

EX NO: 8b	SIMULATION AND ANALYSIS OF QPSK MODULATION USING CONSTELLATION DIAGRAMS IN MATLAB
DATE:	

```
clc;
```

```
clear;
```

```
close all;
```

```
%% Objective 1: Generate QPSK Symbols
```

```
N = 1000;           % Number of bits
```

```
data = randi([0 1], N, 1); % Random binary sequence
```

```
% Ensure even number of bits
```

```
if mod(N,2) ~= 0
```

```
    data = [data; 0];
```

```
    N = N + 1;
```

```
end
```

```
% Group bits into pairs
```

```
bit_pairs = reshape(data, 2, []);
```

```
% QPSK Mapping
```

```
qpsk_symbols = zeros(size(bit_pairs,1),1);
```

```
for k = 1:size(bit_pairs,1)
```

```
    if isequal(bit_pairs(k,:), [0 0])
```

```
        qpsk_symbols(k) = (1 + 1j)/sqrt(2);
```

```
    elseif isequal(bit_pairs(k,:), [0 1])
```

```

    qpsk_symbols(k) = (-1 + 1j)/sqrt(2);

elseif isequal(bit_pairs(k,:), [1 1])
    qpsk_symbols(k) = (-1 - 1j)/sqrt(2);

elseif isequal(bit_pairs(k,:), [1 0])
    qpsk_symbols(k) = (1 - 1j)/sqrt(2);
end

end

%% Objective 2: Plot Ideal QPSK Constellation

figure;

scatter(real(qpsk_symbols), imag(qpsk_symbols), 30, 'filled');

xlim([-1.5 1.5]);
ylim([-1.5 1.5]);

hold on;

xline(0, 'k', 'LineWidth', 1.5);
yline(0, 'k', 'LineWidth', 1.5);

grid on;
axis equal;

```

```

xlabel('In-Phase (I)');
ylabel('Quadrature (Q)');
title('Ideal QPSK Constellation');

%% Objective 3: Add Noise and Evaluate Performance

SNR = 5;           % Signal-to-Noise Ratio in dB

rx_signal = awgn(qpsk_symbols, SNR, 'measured');

%% Plot Noisy QPSK Constellation

figure;

scatter(real(rx_signal), imag(rx_signal), 15, 'filled');

hold on;

xline(0, 'k', 'LineWidth', 1.5);
yline(0, 'k', 'LineWidth', 1.5);

grid on;
axis equal;

xlabel('In-Phase (I)');
ylabel('Quadrature (Q)');

title(['Noisy QPSK Constellation (SNR = ', ...

```

```

num2str(SNR, ' dB']]);

%% Symbol Detection

detected_bits = zeros(length(rx_signal)*2, 1);

for k = 1:length(rx_signal)

    I = real(rx_signal(k));
    Q = imag(rx_signal(k));

    if I > 0 && Q > 0
        bits = [0 0];

    elseif I < 0 && Q > 0
        bits = [0 1];

    elseif I < 0 && Q < 0
        bits = [1 1];

    else
        bits = [1 0];
    end

    detected_bits(2*k-1 : 2*k) = bits;

end

```

%% BER Calculation

```
num_errors = sum(data ~= detected_bits);
```

```
BER = num_errors / N;
```

%% Display Results

```
fprintf('Number of Transmitted Bits : %d\n', N);
```

```
fprintf('Number of Bit Errors    : %d\n', num_errors);
```

```
fprintf('Bit Error Rate (BER)    : %f\n', BER);
```

